



포팅 매뉴얼



태그

인프라

BATT 프로젝트 포팅 매뉴얼

목차

1. 개요

1-1. 프로젝트 개요

1-2. 사용 기술 및 버전

1-3. 시스템 아키텍처

1-4. 포트 구성

1-5. 외부 서비스

2. 환경 설정 및 빌드

2-1. 프로젝트 구조

2-2. 로컬 개발 환경 구축 순서

2-3. 배포 환경 구축 순서

1. 개요

1-1. 프로젝트 개요

BATT(바티, 바른티켓)는 공연 티켓 예매 및 양도 경매 웹사이트입니다.

- 서비스 URL : <https://i13A506.p.ssafy.io/>
- Jenkins admin URL : <http://i13A506.p.ssafy.io:9090/>

1-2. 사용 기술 및 버전

★ Backend

- **Java:** 21
- **Framework:** Spring Boot 3.5.3
- **Build Tool:** Gradle
- **WAS:** 내장 Tomcat (Spring Boot)
- **IDE:** IntelliJ IDEA

★ Frontend

- **Node.js:** v22.17.0(LTS)
- **Framework:** React + Vite + TypeScript
- **Package Manager:** npm (로컬) / yarn (배포 환경)
- **Web Server:** Nginx 1.24+ (Docker 컨테이너)
- **IDE:** Visual Studio Code

★ Database & Cache

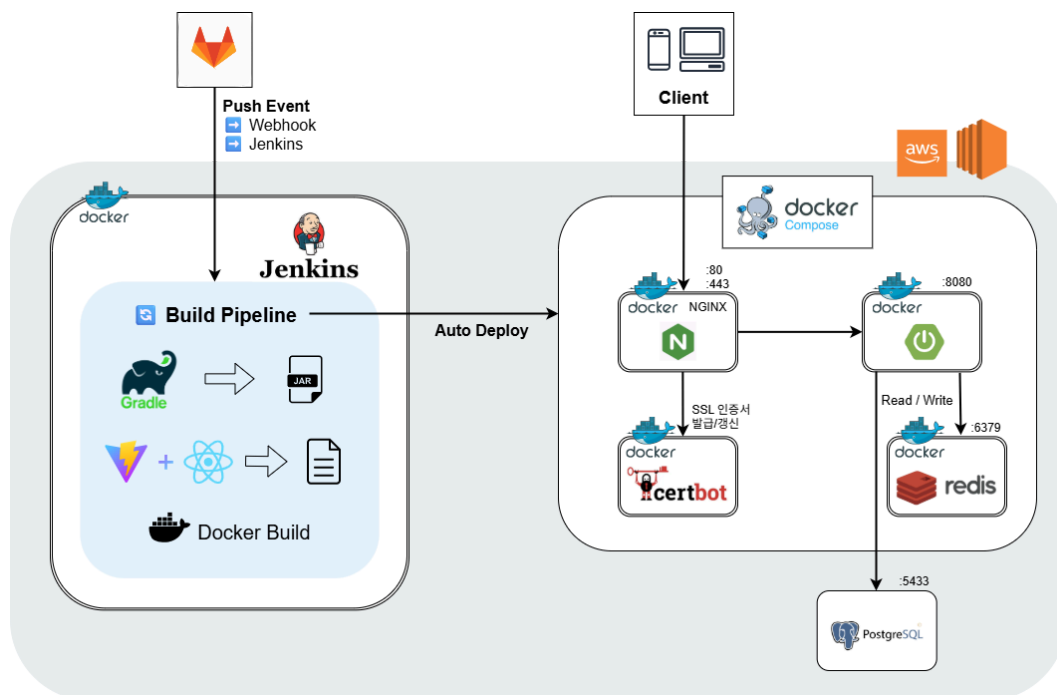
- **Database:** PostgreSQL 17.5 (로컬) / 14 (배포 환경, EC2 직접 설치)

- Cache: Redis 8.0.3-alpine (Docker 컨테이너)

★ Infrastructure

- Server: AWS EC2 Ubuntu 22.04.4 LTS (XLARGE: 4vCPU, 16GB RAM, 320GB SSD)
- **Containerization:** Docker, Docker Compose
- **CI/CD:** Jenkins 2.x (Docker 컨테이너)
- **SSL:** Let's Encrypt + Certbot
- **Domain:** i13a506.p.ssafy.io

1-3. 시스템 아키텍처



1-4. 포트 구성

- **80/443:** HTTPS 리버스 프록시 (외부 접근)

- **8080**: Spring Boot Backend (컨테이너 내부)
- **80**: React Frontend + Nginx (컨테이너 내부)
- **5433**: PostgreSQL (EC2 직접 설치, 외부 접근 허용)
- **6379**: Redis (컨테이너 내부)
- **9090**: Jenkins (외부 접근)

1-5. 외부 서비스

✅ SSL 인증서 : Let's Encrypt

✅ 소셜 로그인 API

• Kakao Developer (필수)

- 사이트: <https://developers.kakao.com>
- 앱 생성 후 설정:
 - 플랫폼 추가: Web (<https://i13a506.p.ssafy.io>)
 - Redirect URI: <https://i13a506.p.ssafy.io/login/oauth2/code/kakao>
 - 동의 항목: 이메일, 이름, 성별, 생일, 출생 연도, 전화번호
- 발급받을 정보: REST API 키, Client Secret

• Naver Developers (필수)

- 사이트: <https://developers.naver.com>
- 애플리케이션 등록:
 - 서비스 URL: <https://i13a506.p.ssafy.io>
 - Callback URL: <https://i13a506.p.ssafy.io/login/oauth2/code/naver>
 - 제공 정보: 이메일, 이름, 성별, 생일, 출생 연도, 전화번호

- 발급받을 정보: Client ID, Client Secret

✅ Naver Maps API (필수)

- 사이트: <https://www.ncloud.com/product/applicationService/maps>
- 용도: 공연장 위치 표시
- 발급받을 정보: Client ID, Client Secret

✅ Discord Webhook (선택)

- 용도: CI/CD 결과 알림
- 설정: Discord 서버 → 채널 → 웹훅 생성
- 발급받을 정보: Webhook URL

2. 환경 설정 및 빌드

2-1. 프로젝트 구조

```
S13P11A506/
├── batt-be/                # Spring Boot Backend
│   ├── src/main/resources/
│   │   ├── application-docker.yml  # 배포 환경 설정
│   │   ├── application-local.yml   # 로컬 개발 설정
│   │   ├── application-secret.yml  # 로컬 민감정보 설정(.gitignore 포함)
│   │   └── logback-spring.xml      # 로깅 설정
│   └── Dockerfile
```

```

|   └─ build.gradle
|   └─ batt-fe/          # React Frontend
|       └─ src/
|       └─ Dockerfile
|       └─ .env          # 로컬 환경변수 설정(.gitignore 포함)
|       └─ nginx.conf    # Nginx 설정
|       └─ package.json
|   └─ batt-m/          # Android (모바일 티켓 구현)
|       └─ local.properties # 로컬 민감정보 설정(.gitignore 포함)
|   └─ docker-compose.yml # 컨테이너 오케스트레이션
|   └─ Jenkinsfile      # CI/CD 파이프라인(jenkins GUI 내부에 저장)

```

2-2. 로컬 개발 환경 구축 순서

✅ 1단계 : 기본 환경 준비

- PostgreSQL, Redis → Docker 컨테이너로 준비 및 연결

```

# docker container → postgresql
docker run --name batt \
  -e POSTGRES_PASSWORD=[민감정보] \
  -e POSTGRES_USER=batt \
  -e POSTGRES_DB=batt \
  -p 5432:5432 \
  -d postgres:17.5

# docker container → redis
docker run --name redis -p 6379:6379 -d redis:8.0.3-alpine

```

✅ 2단계 : Backend 설정

- 로컬 환경설정은 batt-be/src/main/resources/application-local.yml 기준
- 로컬 개발 시 민감정보는 batt-be/src/main/resources/application-secret.yml 에서 관리

- DB 연결 및 Backend 실행의 경우 IDE에서 진행
- IntelliJ 사용 시 Active profiles를 'local'로 설정

```
cd batt-be/src/main/resources
```

```
# application-secret.yml은 .gitignore 항목이므로 별도 생성
vi application-secret.yml
```

- **application-secret.yml 파일**

- 민감정보의 경우 마스킹 처리하였습니다.

```
# batt-be/src/main/resources/application-local.yml
```

```
DATABASE:
```

```
  LOCAL:
```

```
    url: jdbc:postgresql://localhost:5432/batt
```

```
    username: batt
```

```
    password: [민감정보]
```

```
    driver-class-name: org.postgresql.Driver
```

```
REDIS:
```

```
  LOCAL:
```

```
    host: localhost
```

```
    port: 6379
```

```
HOST:
```

```
  ALLOWED-ORIGINS: "http://localhost:8080,http://localhost:5173,http://127.0.0.1:5173,http://127.0.0.1:8080,https://i13a506.p.ssafy.io"
```

```
  FRONTEND: http://localhost:5173
```

```
LOG:
```

```
  PATH: ./batt-be/logs
```

```
  FILENAME:
```

```
    DEFAULT: batt
```

```
    ERROR: batt-error
```

```
OAUTH2:
```

```
  KAKAO:
```

```
AUTHORIZATION-URL: https://kauth.kakao.com/oauth/authorize
TOKEN-URL: https://kauth.kakao.com/oauth/token
USER-INFO-URI: https://kapi.kakao.com/v2/user/me
OIDC-JWK-SET-URI: https://kauth.kakao.com/.well-known/jwks.json
GRANT-TYPE: authorization_code
LOGIN-REDIRECT-URI: http://localhost:8080/login/oauth2/code/kaka
o
CLIENT-ID: [민감정보]
CLIENT-SECRET-KEY: [민감정보]
NAVER:
AUTHORIZATION-URL: https://nid.naver.com/oauth2.0/authorize
TOKEN-URL: https://nid.naver.com/oauth2.0/token
USER-INFO-URI: https://openapi.naver.com/v1/nid/me
GRANT-TYPE: authorization_code
LOGIN-REDIRECT-URI: http://localhost:8080/login/oauth2/code/naver
CLIENT-ID: [민감정보]
CLIENT-SECRET-KEY: [민감정보]

JWT:
SECRET: [민감정보]

COOKIE:
DOMAIN:
LOCAL: localhost
PROD: i13a506.p.ssafy.io
DOCKER: i13a506.p.ssafy.io
SECURE-MODE:
LOCAL: false
PROD: true
DOCKER: true
```

✅ 3단계 : Frontend 설정

- 로컬 개발 시 환경 변수는 batt-fe/.env 에서 진행
- 로컬 개발 시 패키지매니저는 npm 사용


```
# 1. .env 파일 생성 (.gitignore 항목이므로 별도 생성)
cd batt-fe
vi .env
```

```
# 2. frontend 실행
npm install
npm run dev
# http://localhost:5173 접속
```

- **.env 파일**

- `VITE_API_URL` : 백엔드 API 서버 주소 (로컬 개발시 주석 처리하면 localhost 연결)
- 민감정보의 경우 마스킹 처리하였습니다.

```
# batt-fe/.env
VITE_API_URL=https://i13a506.p.ssafy.io
VITE_NAVER_MAP_CLIENT_ID=[민감정보]
VITE_NAVER_MAP_CLIENT_SECRET=[민감정보]
```

✅ 4단계 : Android mobile 설정

- 로컬 개발 시 민감정보 관리는 batt-m/local.properties 에서 진행

```
cd batt-m
vi local.properties    #.gitignore 항목이므로 별도 생성
```

- **local.properties 파일**

- 민감정보의 경우 마스킹 처리하였습니다.

```
sdk.dir=C:\\Users\\SSAFY\\AppData\\Local\\Android\\Sdk
NATIVE_APP_KEY=[민감정보]
NAVER_CLIENT_ID=[민감정보]
```

```
NAVER_CLIENT_SECRET_KEY=[민감정보]
TOSS_CERT_ACCESS_TOKEN=[민감정보]
```

2-3. 배포 환경 구축 순서

✅ 1단계 : EC2 서버 초기 설정

- SSAFY에서 제공받은 EC2
- 기본 시스템 업데이트

```
sudo apt update && sudo apt upgrade -y
```

• Docker & Docker Compose 설치

```
# Docker 공식 GPG 키 추가
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --
dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
```

```
# Docker 저장소 추가
echo "deb [arch=amd64 signed-by=/usr/share/keyrings/docker-archiv
e-keyring.gpg] https://download.docker.com/linux/ubuntu $(lsb_releas
e -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
# Docker 설치
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io -y
```

```
# 현재 사용자를 docker 그룹에 추가 (sudo 없이 docker 명령어 사용)
sudo usermod -aG docker $USER
```

```
# Docker Compose 설치 (최신 버전)
sudo curl -L "https://github.com/docker/compose/releases/latest/down
```

```
load/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
```

```
# 실행 권한 부여
```

```
sudo chmod +x /usr/local/bin/docker-compose
```

- **포트 및 방화벽 설정 추가**

```
# UFW 방화벽 설정
```

```
sudo ufw allow 22      # SSH
```

```
sudo ufw allow 80      # HTTP (Nginx)
```

```
sudo ufw allow 443     # HTTPS (SSL)
```

```
sudo ufw allow 5433    # PostgreSQL
```

```
sudo ufw allow 9090    # Jenkins
```

```
# UFW 상태 확인
```

```
sudo ufw status
```

```
# UFW 활성화 (비활성화 상태일 때)
```

```
sudo ufw enable
```

```
# 설정 확인
```

```
sudo ufw status numbered
```

- **DB 설치 및 설정**

- PostgreSQL은 Docker로 올리지 않고 EC2에 직접 설치 (데이터 영속성)
- 구축 진행 시 포트 5433번으로 자동 설정되어, 포트 5433번으로 사용했었음.

```
# DB 설치
```

```
apt install postgresql postgresql-contrib -y
```

```
# EC2 PostgreSQL에서 사용자 생성
```

```
sudo -u postgres psql -p 5433
```

```
# EC2 PostgreSQL에서 사용자 생성 SQL문
CREATE USER batt WITH PASSWORD [비밀번호];
CREATE DATABASE batt OWNER batt;
GRANT ALL PRIVILEGES ON DATABASE batt TO batt;
\q
```

◦ PostgreSQL 외부 접속 허용 설정

```
# postgresql.conf 수정
sudo nano /etc/postgresql/14/main/postgresql.conf
# listen_addresses = '*' 로 변경
```

```
# - Connection Settings -
listen_addresses = '*'          # what IP address(es) to listen on;
                                # comma-separated list of addresses;
                                # defaults to 'localhost'; use '*' for all
                                # (change requires restart)
```

```
# pg_hba.conf 수정 - 실제로 연결 허용할 네트워크 추가
sudo nano /etc/postgresql/14/main/pg_hba.conf
```

```
# 다음 줄 추가:
#host    all    all    172.17.0.0/16   md5 # Docker 기본 bridge 네트워크
#host    all    all    172.18.0.0/16   md5 # Docker 추가 네트워크 (compose
등)
#host    all    all    172.26.0.0/16   md5 # 현재 EC2 내부 네트워크
#host    batt   batt   172.25.0.0/16   md5 # BATT 프로젝트 전용 네트워크
```

host	all	all	172.17.0.0/16	md5
host	all	all	172.18.0.0/16	md5
host	all	all	172.26.0.0/16	md5
host	batt	batt	172.25.0.0/16	md5

```
# 설정 변경 후 PostgreSQL 재시작
sudo systemctl restart postgresql
sudo systemctl enable postgresql # 부팅시 자동 시작
```

```
# 상태 확인
sudo systemctl status postgresql

# EC2에서 DB 연결 테스트
psql -h localhost -p 5433 -U batt -d batt
```

✅ 2단계 : SSL/HTTPS 설정

- SSL 인증서 저장소 생성

```
# SSL 인증서 영구 저장 폴더 생성
cd /home/ubuntu
mkdir -p ssl/certbot/conf ssl/certbot/www
```

- Let's Encrypt 인증서 발급

```
docker run --rm \
-v /home/ubuntu/ssl/certbot/conf:/etc/letsencrypt \
-v /home/ubuntu/ssl/certbot/www:/var/www/certbot \
certbot/certbot certonly --webroot \
-w /var/www/certbot \
--email [이메일주소] \
--agree-tos \
--non-interactive \
-d i13a506.p.ssafy.io
```

- 인증서 경로 확인 및 SSL 서비스 시작

```
# 발급된 인증서 경로 확인 (nginx.conf에서 실제 경로로 수정 필요)
ls -la /home/ubuntu/ssl/certbot/conf/live/

# docker-compose로 SSL서비스 시작 (Jenkins CI/CD 과정에 포함)
```

```
# HTTPS 작동 확인
curl -I https://i13a506.p.ssafy.io
```

✅ 3단계 : Jenkins 구축

- Jenkins CI/CD 파이프라인을 통해 자동으로 gitlab 브랜치의 코드가 배포되도록 설정
- 환경변수는 application-docker.yml, docker-compose.yml, Jenkinsfile, Jenkins Credentials를 통해 관리됨.
- CI/CD 파이프라인 구축 후 '지금 빌드' 실행 시 자동 빌드됨
- EC2 내에서 jenkins 컨테이너 실행 및 jenkins 내 docker-compose 설치

```
# jenkins 컨테이너 실행
docker run -d \
  --name jenkins \
  -p 9090:8080 \
  -e TZ=Asia/Seoul \
  -e JAVA_OPTS="-Duser.timezone=Asia/Seoul -Xmx2g -Xms512m" \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v /usr/bin/docker:/usr/bin/docker \
  --user root \
  --restart unless-stopped \
  --memory=4g \
  --memory-swap=4g \
  jenkins/jenkins:lts

# docker-compose 설치
docker exec jenkins sh -c "
  curl -L https://github.com/docker/compose/releases/latest/download/
  docker-compose-Linux-x86_64 -o /usr/local/bin/docker-compose &&
  chmod +x /usr/local/bin/docker-compose"
```

```
# docker-compose 설치여부 및 버전 확인
docker exec jenkins docker-compose --version

# 초기 패스워드 확인
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

- 브라우저 접속 → 설정
 - <http://i13a506.p.ssafy.io:9090/>
 - 플러그인 설치 (Git, Pipeline, Docker, Credentials, Gradle, NodeJS)
 - 관리자 계정 생성, 플러그인 별 설정 진행
- New Item → Pipeline 생성
 - Configuration 설정



- Enable : 켜두기
- 설명 : BATT 프로젝트 자동 빌드 파이프라인 (추후 수정 가능)
- 오래된 빌드 삭제 : 체크 (선택사항)
 - 빌드 이력 유지 기간(일) : 30
 - 보관할 최대 갯수 : 20
- Pipeline
 - Definition : Pipeline script
 - Script : 작성해둔 Jenkinsfile 입력
 - Use Groovy Sandbox : 체크
- 그 외 사항 → 체크 해제 (추후 GitLab Webhook 설정 등 추가 예정)
- **Save** 선택! (Save는 설정 저장 후 Job 대시보드로 돌아감 / Apply는 중간저장 역할)

- 민감한 환경변수 Jenkins Credentials에 등록
 - 최종 Jenkins Credentials 목록

Credentials

T	P	Store ↓	Domain	ID	Name
		System	(global)	gitlab-batt-token	(GitLab BATT Project Access Token)
		System	(global)	postgres-password	PostgreSQL password for dev environment
		System	(global)	jwt-secret	JWT 토큰 서명을 위한 비밀키
		System	(global)	kakao-client-id	카카오 OAuth 2.0 애플리케이션 클라이언트 ID
		System	(global)	kakao-client-secret	카카오 OAuth 2.0 애플리케이션 클라이언트 시크릿 키
		System	(global)	naver-client-id	네이버 OAuth 2.0 애플리케이션 클라이언트 ID
		System	(global)	naver-client-secret	네이버 OAuth 2.0 애플리케이션 클라이언트 시크릿 키
		System	(global)	naver-map-client-id	네이버맵 API 클라이언트 ID (프론트엔드용)
		System	(global)	naver-map-client-secret	네이버맵 API 클라이언트 시크릿 (프론트엔드용)
		System	(global)	discord-webhook-url	Discord Webhook for CI/CD notifications

- Push event 시 자동 배포를 위해 gitlab Webhook 설정 진행
 - URL:** <http://i13A506.p.ssafy.io:9090/project/BATT-Pipeline>
 - 트리거:** Push to dev branch
 - 인증:** Personal Access Token

- Jenkinsfile

```

pipeline {
  agent any

  tools {
    nodejs 'Node22'
  }
}

```



```

environment {
    DOCKER_IMAGE_BACKEND = "batt-backend"
    DOCKER_IMAGE_FRONTEND = "batt-frontend"
    BUILD_NUMBER = "${env.BUILD_NUMBER}"
    COMPOSE_PROJECT_NAME = "batt"

    // Discord 알림용
    DISCORD_WEBHOOK = credentials('discord-webhook-url')

    // DB 관련 (외부 DB 사용하니까 필요)
    POSTGRES_DB = 'batt'
    POSTGRES_USER = 'batt'

    // 🗝 Jenkins Credentials (보안 필수)
    POSTGRES_PASSWORD = credentials('postgres-password')
    JWT_SECRET = credentials('jwt-secret')
    OAUTH2_KAKAO_CLIENT_ID = credentials('kakao-client-id')
    OAUTH2_KAKAO_CLIENT_SECRET = credentials('kakao-client-secret')
    OAUTH2_NAVER_CLIENT_ID = credentials('naver-client-id')
    OAUTH2_NAVER_CLIENT_SECRET = credentials('naver-client-secret')
    NAVER_MAP_CLIENT_ID = credentials('naver-map-client-id')
    NAVER_MAP_CLIENT_SECRET = credentials('naver-map-client-secret')

    // Redirect URI (도메인 변경 가능성)
    OAUTH2_KAKAO_LOGIN_REDIRECT_URI = 'https://i13a506.p.ssafy.io/login/oauth2/code/kakao'
    OAUTH2_NAVER_LOGIN_REDIRECT_URI = 'https://i13a506.p.ssafy.io/login/oauth2/code/naver'

    // Spring Boot 설정
    SPRING_PROFILES_ACTIVE = 'docker'
    SPRING_DATASOURCE_URL = "jdbc:postgresql://172.17.0.1:5433/${POSTGRES_DB}"
    SPRING_DATASOURCE_USERNAME = "${POSTGRES_USER}"
    SPRING_DATASOURCE_PASSWORD = "${POSTGRES_PASSWORD}"

```

```

// 호스트 설정
HOST_ALLOWED_ORIGINS = 'https://i13a506.p.ssafy.io,http://localhost:5173,http://localhost:8080,http://127.0.0.1:5173,http://127.0.0.1:8080'
HOST_FRONTEND = 'https://i13a506.p.ssafy.io'

// 쿠키 설정
COOKIE_DOMAIN_DOCKER = 'i13a506.p.ssafy.io'
COOKIE_SECURE_MODE_DOCKER = 'true'

// 프론트엔드 빌드용
VITE_NAVER_MAP_CLIENT_ID = "${NAVER_MAP_CLIENT_ID}"
VITE_NAVER_MAP_CLIENT_SECRET = "${NAVER_MAP_CLIENT_SECRET}"
T}"
}

stages {
  stage('Git 체크아웃') {
    steps {
      echo '=== GitLab에서 소스코드 가져오기 (실제 Docker 파일 포함) ==='

      git branch: 'dev',
        url: 'https://lab.ssafy.com/s13-webmobile2-sub1/S13P11A506.git',
        credentialsId: 'gitlab-batt-token'
      sh 'pwd && ls -la'

      echo '=== Docker 파일 존재 확인 ==='
      sh 'ls -la batt-be/Dockerfile'
      sh 'ls -la batt-fe/Dockerfile'
      sh 'ls -la docker-compose.yml'
      sh 'echo "✅ 모든 Docker 파일 확인 완료!'"
    }
  }

  stage('백엔드 빌드') {
    steps {
      echo '=== Spring Boot 빌드 ==='
      dir('batt-be') {

```

```

        sh 'chmod +x ./gradlew'
        sh './gradlew clean build -x test'
        sh 'ls -la build/libs/'
    }
}
}

stage('프론트엔드 빌드') {
    steps {
        echo '=== React + Vite 빌드 (Yarn 사용) ==='
        dir('batt-fe') {
            echo '1. 환경 확인'
            sh 'node --version && yarn --version || npm install -g yarn'

            echo '2. 환경변수 설정, 의존성 설치 및 빌드'
            sh '''
                # 환경변수 설정
                export VITE_API_URL=https://i13a506.p.ssafy.io
                export VITE_NAVER_MAP_CLIENT_ID="${VITE_NAVER_MAP_
CLIENT_ID}"
                export VITE_NAVER_MAP_CLIENT_SECRET="${VITE_NAVER
_MAP_CLIENT_SECRET}"

                # 기존 파일 정리
                rm -rf node_modules package-lock.json yarn.lock

                # Yarn으로 의존성 설치
                yarn install

                # 빌드 실행 (환경변수 유지됨)
                yarn build
            '''

            echo '3. 빌드 결과 확인'
            sh 'ls -la dist/'
        }
    }
}
}

```

```

stage('Dockerfile 내용 확인') {
    steps {
        echo '=== 실제 Dockerfile 내용 확인 ==='
        sh 'echo "=== 백엔드 Dockerfile ===" && cat batt-be/Dockerfile'
        sh 'echo "=== 프론트엔드 Dockerfile ===" && cat batt-fe/Dockerfile'

        sh 'echo "=== Docker Compose ===" && head -20 docker-compose.yml'
    }
}

stage('환경변수 확인') {
    steps {
        echo '=== Jenkins Credentials 환경변수 확인 ==='
        sh 'echo "POSTGRES_DB: ${POSTGRES_DB}"'
        sh 'echo "POSTGRES_USER: ${POSTGRES_USER}"'
        sh 'echo "Spring DataSource URL: ${SPRING_DATASOURCE_URL}"'
        sh 'echo "Spring DataSource Username: ${SPRING_DATASOURCE_USERNAME}"'
        sh 'echo "Redis Host: ${SPRING_DATA_REDIS_HOST}"'
        sh 'echo "비밀번호는 보안상 마스킹됩니다"'
    }
}

stage('Docker 이미지 빌드') {
    parallel {
        stage('백엔드 이미지') {
            steps {
                echo '=== 백엔드 Docker 이미지 빌드 (실제 Dockerfile 사용) ==='

                dir('batt-be') {
                    sh "docker build -t ${DOCKER_IMAGE_BACKEND}:${BUILD_NUMBER} ."
                    sh "docker tag ${DOCKER_IMAGE_BACKEND}:${BUILD_NUMBER} ${DOCKER_IMAGE_BACKEND}:latest"
                    sh "echo '✅ 백엔드 이미지 빌드 완료'"
                }
            }
        }
    }
}

```

```

    }
  }
}
stage('프론트엔드 이미지') {
  steps {
    echo '=== 프론트엔드 Docker 이미지 빌드 (실제 Dockerfile 사용)
=== '

    dir('batt-fe') {
      sh "docker build -t ${DOCKER_IMAGE_FRONTEND}:${BUILD_
LD_NUMBER} ."
      sh "docker tag ${DOCKER_IMAGE_FRONTEND}:${BUILD_
NUMBER} ${DOCKER_IMAGE_FRONTEND}:latest"
      sh "echo '✅ 프론트엔드 이미지 빌드 완료'"
    }
  }
}

stage('이미지 확인') {
  steps {
    echo '=== 생성된 Docker 이미지 확인 ==='
    sh 'docker images | grep batt'
    sh 'docker images | grep batt | wc -l | xargs echo "생성된 BATT 이
미지 개수:"'
  }
}

stage('기존 서비스 완전 정리') {
  steps {
    echo '=== 기존 풀스택 서비스 완전 정리 ==='
    script {
      // 강화된 정리 프로세스
      sh 'docker-compose -p ${COMPOSE_PROJECT_NAME} down -
-remove-orphans || true'
      sh '''
        docker stop certbot spring_backend react_frontend_ssl redis
_cache || true

```

```

        docker rm certbot spring_backend react_frontend_ssl redis_c
ache || true
        docker container prune -f
        docker network prune -f
        ""
        echo '✅ 기존 서비스 완전 정리 완료'
    }
}

stage('풀스택 서비스 배포') {
    steps {
        echo '=== Jenkins 빌드 이미지로 풀스택 배포 ==='
        script {
            // Jenkins가 빌드한 이미지 확인
            sh 'docker images | grep batt'

            // docker-compose 실행 (Jenkins 빌드 이미지 사용)
            sh 'docker-compose -p ${COMPOSE_PROJECT_NAME} up -d'
            sh 'sleep 25' // DB 초기화 + 백엔드 시작 시간
            sh 'docker-compose -p ${COMPOSE_PROJECT_NAME} ps'
            echo '✅ 풀스택 서비스 배포 완료'
        }
    }
}

stage('배포 검증') {
    steps {
        echo '=== 서비스 배포 확인 ==='
        script {
            sh 'docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Port
s}}"'

            sh 'echo "✅ 모든 서비스 배포 완료!"'
            sh 'echo "🌐 서비스 접속: https://i13A506.p.ssafy.io/'
        }
    }
}

```

```
}  
}
```

- +) Discord Webhook 알림을 위한 추가 코드 → stages 다음 코드로 추가

```
post {  
    success {  
        script {  
            // 기존 성공 로직  
            echo '🎉 완전한 CI/CD 파이프라인 성공!'  
            echo '✅ 빌드 → 이미지 생성 → 자동 배포 완료!'  
            echo '🌐 서비스 접속: https://i13A506.p.ssafy.io/'  
            sh 'docker images | grep batt || echo "이미지 없음"'   
  
            // Discord 알림 로직  
            try {  
                def triggeredBy = "직접 빌드"  
                def pushUser = "알 수 없음"  
  
                if (env.gitlabUserName) {  
                    triggeredBy = "GitLab Push"  
                    pushUser = env.gitlabUserName  
                } else {  
                    def userCauses = currentBuild.getBuildCauses('hudson.  
model.Cause$UserIdCause')  
                    if (userCauses && userCauses.size() > 0) {  
                        triggeredBy = "수동 빌드"  
                        pushUser = userCauses[0].userId ?: "admin"  
                    }  
                }  
  
                def commitAuthor = "알 수 없음"  
                def commit = "unknown"  
                try {  
                    commitAuthor = sh(returnStdout: true, script: 'git log -1 --  
pretty=format:"%an" 2>/dev/null || echo "알 수 없음"').trim()  
                }  
            }  
        }  
    }  
}
```

```

        commit = sh(returnStdout: true, script: 'git rev-parse --sh
ort HEAD 2>/dev/null || echo "unknown").trim()
    } catch (Exception gitError) {
        echo "Git 정보 수집 실패: ${gitError.message}"
    }

    if (env.DISCORD_WEBHOOK) {
        echo "🚀 Discord 성공 알림 전송 중..."
        sh """
            timeout 10 curl -H "Content-Type: application/json" \
                -X POST \
                -d '{"content": "✅ **BATT 배포 성공!** 🎉\n🚀 **
${triggeredBy}**: ${pushUser}\\n📝 커밋 작성자: ${commitAuthor} | 📦
# ${BUILD_NUMBER} | 🔗 ${commit}\\n🌐 https://i13a506.p.ssafy.io}"' \
                "${env.DISCORD_WEBHOOK}" || echo "Discord 알림
실패 (무시됨)"
            """
        } else {
            echo "Discord Webhook이 설정되지 않음 - 알림 건너뛰"
        }

    } catch (Exception e) {
        echo "Discord 알림 처리 중 오류 발생: ${e.message}"
        echo "빌드는 성공적으로 완료되었습니다."
    }
}

archiveArtifacts artifacts: 'batt-be/build/libs/*.jar, batt-fe/dist/**/
*',
    allowEmptyArchive: true
}

failure {
    script {
        // 기존 실패 로직
        echo '❌ 배포 실패!'
        echo '디버깅 정보:'
        sh 'pwd && ls -la'
    }
}

```



```

sh 'docker images | tail -10 || true'
sh 'docker-compose -p ${COMPOSE_PROJECT_NAME} logs |
| true'

// Discord 실패 알림
try {
  def triggeredBy = "직접 빌드"
  def pushUser = "알 수 없음"

  if (env.gitlabUserName) {
    triggeredBy = "GitLab Push"
    pushUser = env.gitlabUserName
  } else {
    def userCauses = currentBuild.getBuildCauses('hudson.
model.Cause$UserIdCause')
    if (userCauses && userCauses.size() > 0) {
      triggeredBy = "수동 빌드"
      pushUser = userCauses[0].userId ?: "admin"
    }
  }

  def commitAuthor = "알 수 없음"
  def commit = "unknown"
  try {
    commitAuthor = sh(returnStdout: true, script: 'git log -1 --
pretty=format:"%an" 2>/dev/null || echo "알 수 없음").trim()
    commit = sh(returnStdout: true, script: 'git rev-parse --sh
ort HEAD 2>/dev/null || echo "unknown").trim()
  } catch (Exception gitError) {
    echo "Git 정보 수집 실패: ${gitError.message}"
  }

  if (env.DISCORD_WEBHOOK) {
    echo "🚀 Discord 실패 알림 전송 중..."
    sh """
      timeout 10 curl -H "Content-Type: application/json" \
        -X POST \
        -d '{"content": "❌ **BATT 배포 실패** 🚀\\n🚀" **$t

```

```

triggeredBy}**: ${pushUser}\\n📝 커밋 작성자: ${commitAuthor} | 1 2 3 4 #
${BUILD_NUMBER} | 🔗 ${commit}\\n🔍 [로그보기](${BUILD_URL}console)"' \
                                "${env.DISCORD_WEBHOOK}" || echo "Discord 알림
실패 (무시됨)"
        """
    } else {
        echo "Discord Webhook이 설정되지 않음 - 알림 건너뛰"
    }

} catch (Exception e) {
    echo "Discord 알림 처리 중 오류 발생: ${e.message}"
    echo "빌드 실패는 기록되었습니다."
}
}
}
}
}

```

- Pipeline 동작 정리
 1. GitLab dev 브랜치 Push 감지
 2. 소스코드 체크아웃
 3. Backend/Frontend 빌드
 4. Docker 이미지 생성
 5. 컨테이너 재배포